Advanced CSS Techniques

1- CSS custom properties (variables)

2- Smooth transitions

3- Subtle animations

4- Pseudo-classes & pseudo-elements

5- Transform effects

Level up your design!

SCROLL

Quick Recap

Last Class: Responsive Design Deep Dive

RESPONSIVE IMAGES:

- max-width: 100%; height: auto;
- srcset (multiple image sizes)
- picture element (different crops)
- object-fit (cover, contain)
- aspect-ratio (16/9, 1/1, 4/3)



FLUID TYPOGRAPHY:

- clamp(min, preferred, max)
- vw units for scaling text
- ch unit (65ch for readability)
- CSS variables with clamp()

CONTAINER QUERIES:

- container-type: inline-size
- @container (min-width: 400px)
- Query component size, not viewport
- Truly reusable components

RESPONSIVE NAVIGATION:

- Simple stack (column to row)
- Hamburger menu pattern
- Checkbox trick (no JavaScript!)

CSS CUSTOM PROPERTIES



The Variable Problem

Two Layout Systems - Both Essential!

The Problem:

Want to change your primary blue color?

- Find all 27 instances of `#3498db`
- Replace each one manually
- Hope you didn't miss any
- Hope you didn't accidentally change the wrong value
- 20+ minutes of tedious work

Multiple developers?

- Everyone needs to remember the exact color value
- #3498db? Or was it #3499db? Or #3597db?
- Inconsistencies creep in

Want to try different colors?

- Change 27 values to test new color
- Don't like it? Change 27 values back
- Trying 5 options = 135 changes

Hard to maintain:

- No single source of truth
- Values scattered throughout CSS
- Easy to introduce bugs
- Tedious and error-prone

The same applies to:

- Spacing values (1rem, 2rem, etc.)
- Font families
- Font sizes
- Border radius
- Shadows
- Transitions
- ANY repeated value!

CSS Variables to the Rescue

CSS Variables - Change Once, Update Everywhere!

The Benefits:

- **Single Source of Truth** One place to define each value
- **Easy Maintenance** Change once, updates everywhere automatically
- **Self-Documenting** `var(--primary-color)` is clearer than `#3498db`
- **Consistency** Impossible to have accidental variations
- **Fast Iteration** Try different designs in seconds
- **Team-Friendly** Everyone uses the same values
- **No More Find-and-Replace** Never again!

```css
/* 1. Define ONCE in :root */
:root {
  --primary-color: #3498db;
}

/* 2. Use EVERYWHERE with var() */
.header {
  background-color: var(--primary-color);
}

.button {
  background-color: var(--primary-color);
  border: 2px solid var(--primary-color);
}

.link {
  color: var(--primary-color);
}

.link:hover {
  border-bottom: 2px solid var(--primary-color);
}

.border {
  border-color: var(--primary-color);
}

.badge {
  background-color: var(--primary-color);
}

.icon {
  fill: var(--primary-color);
}
```

Variables work for EVERYTHING

```css
/* Spacing */
:root {
  --space-sm: 0.5rem;
  --space-md: 1rem;
  --space-lg: 2rem;
}

.section { padding: var(--space-lg); }
.card { padding: var(--space-md); }
.button { padding: var(--space-sm) var(--space-md); }
```

```css
/* Typography */
:root {
  --font-sans: 'Inter', sans-serif;
  --font-size-xl: 2rem;
  --font-size-base: 1rem;
}

body { font-family: var(--font-sans); }
h1 { font-size: var(--font-size-xl); }
p { font-size: var(--font-size-base); }
```

```css
/* Shadows */
:root {
  --shadow-md: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.card { box-shadow: var(--shadow-md); }
.modal { box-shadow: var(--shadow-md); }
```

Step 1: DECLARE (Define the Variable)

```
:root { --variable-name: value; }
```

DECLARATION:

```
:root {
      --primary-color: #3498db;
      --spacing: 1rem;
      --font-main: 'Arial', sans-serif;
}
```

The Parts:

1. **:root** - Where to define (usually) • `:root` = highest level (html element) • Variables cascade down to all children • Available everywhere in your CSS
2. **--** (Two Dashes) - Variable prefix • REQUIRED! Identifies as custom property • `--primary-color)`
3. **variable-name** - Your chosen name • Use kebab-case (dashes between words) • Descriptive names are best • Case-sensitive! (`--Primary` ≠ `--primary`)
4. **:** and **;** - Standard CSS syntax • Colon after name • Semicolon at end

Step 2: USE (Reference the Variable)

```
.element { property: var(--variable-name); }
```

USAGE:

```
.button {
      background-color: var(--primary-color);
      padding: var(--spacing);
      font-family: var(--font-main);
}
```

The Parts:

1. **var()** - Function to use variable • `var()` retrieves the value • Takes variable name as argument • Returns the defined value
2. **--variable-name** - Reference the variable • Must match declaration exactly • Include the two dashes • Case-sensitive

CSS Variables to the Rescue

The Flow:

1. Browser reads :root
2. Stores variables in memory
3. When it encounters var(--primary-color)
4. Looks up --primary-color
5. Finds: #3498db
6. Uses that value

```
:root {
      --color: #3498db; ✓ Colors
      --size: 1rem; ✓ Sizes
      --family: 'Arial', sans; ✓ Font families
      --weight: bold; ✓ Keywords –
      -shadow: 0 2px 4px rgba(); ✓ Complex values
      --url: url('image.jpg'); ✓ URLs
      --number: 3; ✓ Numbers
}
```

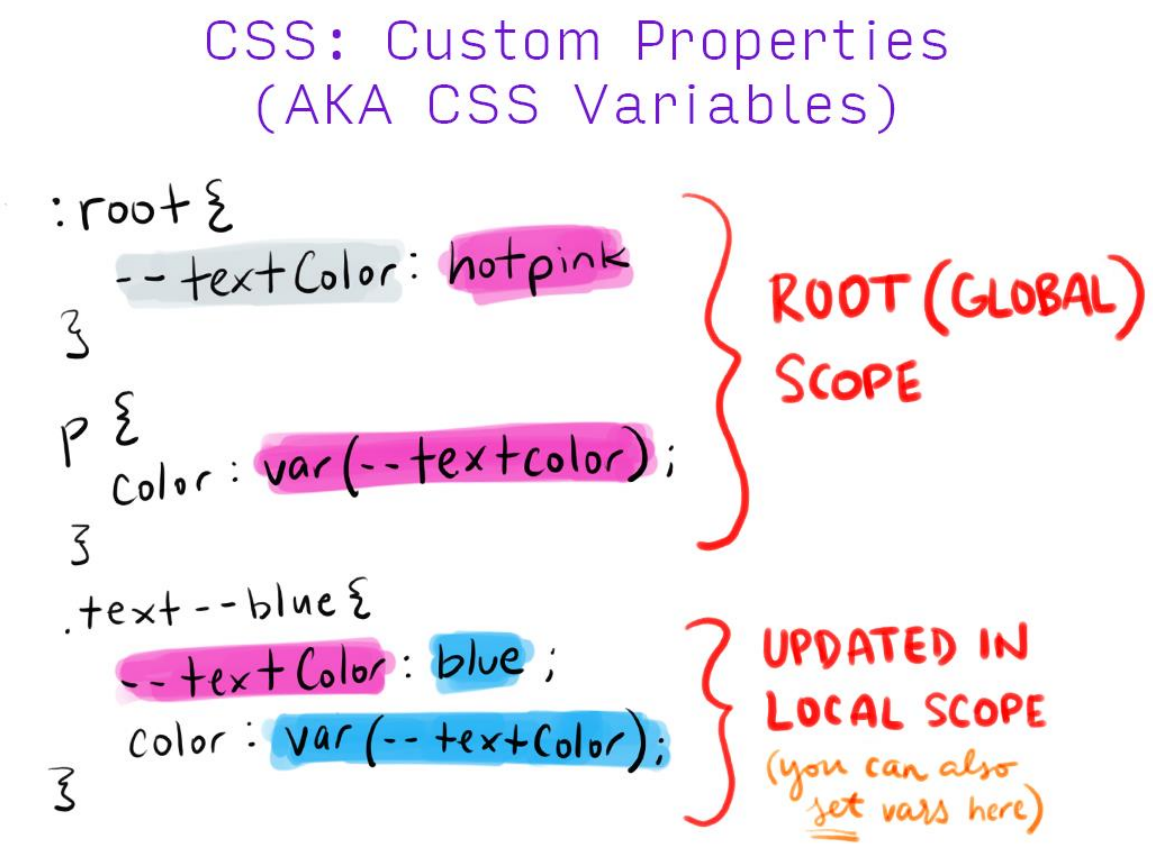
```css
/* ===== STEP 1: DECLARE ===== */
:root {
  --primary-color: #3498db;    /* Define */
  --secondary-color: #2ecc71;
  --text-color: #2c3e50;
  --spacing-small: 0.5rem;
  --spacing-large: 2rem;
}

/* ===== STEP 2: USE ===== */
.header {
  background-color: var(--primary-color);  /* Use */
  color: var(--text-color);
  padding: var(--spacing-large);
}

.button {
  background-color: var(--secondary-color);  /* Use */
  padding: var(--spacing-small) var(--spacing-large);
}

.text {
  color: var(--text-color);  /* Use */
}
```

Benefits of CSS Variables -
Why They're Essential



The Benefits:

- Change values in one place
- Easier to maintain
- Self-documenting code
- Can change with media queries
- Can manipulate with JavaScript
- Create themes easily

The Bottom Line:

Without Variables:

- Hard-coded values scattered everywhere
- Tedious find-and-replace for changes
- Inconsistencies creep in
- Slow iteration and updates
- Difficult collaboration
- Outdated practice

With Variables:

- Single source of truth
- Change once, update everywhere
- Perfect consistency
- Fast iteration
- Easy collaboration
- Professional standard
- Future-proof

CSS TRANSITIONS



What Are Transitions? The Difference Maker

CSS Transitions - Making Websites Feel Alive

What Are Transitions?

- Smooth changes between CSS property values
- Makes interactions feel polished and professional
- The difference between amateur and professional sites
- Incredibly easy to add!

Add ONE line of CSS:

transition: property duration;

```css
.button {
  background-color: #3498db;
  transition: background-color 0.3s ease;  /* ← Magic line! */
}

.button:hover {
  background-color: #2980b9;
}
```

Why Transitions Matter:

1. **Perceived Quality**
   - Without transitions = Amateur, rushed
   - With transitions = Professional, polished
2. **User Feedback**
   - Without: Did something happen?
   - With: Clear acknowledgment of interaction
3. **Emotional Impact**
   - Without: Cold, mechanical
   - With: Warm, responsive, alive
4. **Attention Direction**
   - Smooth transitions guide user's eye
   - Help users understand cause and effect

Transitions: Real-World Examples:

```
/* Button Click: */
/* No transition - instant, jarring */
.button:active {
  transform: scale(0.95);
}

/* With transition - smooth press */
.button {
  transition: transform 0.1s ease;
}

.button:active {
  transform: scale(0.95);
}
```

```
/* Card Hover: */
/* No transition - jumps up suddenly */
.card:hover {
  transform: translateY(-10px);
}

/* With transition - smooth lift */
.card {
  transition: transform 0.3s ease;
}

.card:hover {
  transform: translateY(-10px);
}
```

Transitions: Real-World Examples:

```css
/* Link Hover: */
/* No transition - instant color change */
.link:hover {
  color: #e74c3c;
}

/* With transition - smooth color shift */
.link {
  transition: color 0.2s ease;
}

.link:hover {
  color: #e74c3c;
}
```

Almost any visual CSS property:

- Colors (background, text, border)
- Sizes (width, height, padding)
- Position (top, left, right, bottom)
- Transform (translate, scale, rotate)
- Opacity
- Box-shadow
- Border-radius
- Font-size

Transition Syntax - How to Write Them

CSS Transition Syntax - The Four Parts

```
.element {
  transition: property duration timing-function delay;
}
```

Breaking Down Each Part:

1. **property** - What to animate
2. **duration** - How long it takes
3. **timing-function** - Speed curve (optional)
4. **delay** - Wait before starting (optional)

1. **property** - What to animate

```
/* Part 1: Property (What to Animate) */
.button {
  transition: background-color 0.3s;
}
```

What it means: "Transition the background-color property"

2. **duration** - How long it takes

```
/* Part 2: Duration (How Long) */
.button {
  transition: background-color 0.3s;
  /*                           ↑
                          duration */
}
```

Units:

- **Seconds:** `0.3s`, `1s`, `2.5s`
- **Milliseconds:** `300ms`, `1000ms`, `2500ms`

Transition Syntax - How to Write Them

3. **timing-function -** Speed curve (optional)

```
/* Part 3: Timing Function (Speed Curve) - OPTIONAL */
.button {
  transition: background-color 0.3s ease;
  /*                                 ↑
    | | | | | | | | |              timing function */
}
```

Options:

- **ease (Default):** Most common, (Slow start, Fast middle, Slow end, Feels natural!)
- **linear - Constant speed** (Same speed throughout, Robotic feel, Use for simple animations)
- **ease-in - Slow start, fast end** (Accelerates Good for exits)
- **ease-out - Fast start, slow end** (Decelerates Good for entrances)

4. **delay -** Wait before starting (optional)

```
/* Part 4: Delay (Wait Before Starting) - OPTIONAL */
.button {
  transition: background-color 0.3s ease 0.1s;
  /*                                     ↑
    | | | | | | | | | | | | | | | | | | delay */
}
```

What it does: Waits before starting transition

Transition All Properties

all = transition everything that changes

```css
.card {
  transition: all 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 30px rgba(0,0,0,0.2);
  /* Both will animate smoothly */
}
```

Transition — Common Patterns

Button Hover

```css
/* Button Hover: */
.button {
  background-color: #3498db;
  transform: translateY(0);
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  transition: all 0.3s ease;
}

.button:hover {
  background-color: #2980b9;    /* Color */
  transform: translateY(-2px);     /* Transform */
  box-shadow: 0 4px 8px rgba(0,0,0,0.2); /* Shadow */
}
```

Card Hover

```css
/* Card Hover: */
.card {
  transform: scale(1);
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: scale(1.05);    /* Transform */
  box-shadow: 0 10px 30px rgba(0,0,0,0.2); /* Shadow */
}
```

Transition — Common Patterns

Fade In/Out

```css
/* Fade In/Out: */
.element {
  opacity: 0;
  visibility: hidden;
  transition: opacity 0.3s ease, visibility 0.3s ease;
}

.element.visible {
  opacity: 1;           /* Opacity */
  visibility: visible;  /* Visibility */
}
```

Card Hover

```css
/* Link Underline: */
.link {
  color: #3498db;
  border-bottom: 2px solid transparent;
  transition: color 0.2s ease, border-bottom-color 0.2s ease;
}

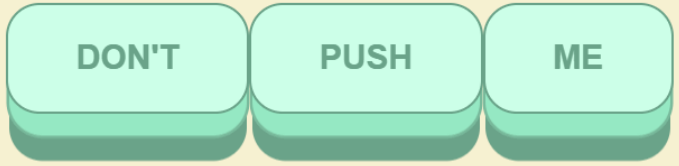
.link:hover {
  color: #2980b9;                /* Color */
  border-bottom-color: #2980b9; /* Border color */
}
```

CSS ANIMATIONS



Transitions vs Animations - Understanding the Difference

Feature	Transitions	Animations
Trigger	User action required	Automatic
States	2 (start, end)	Multiple (keyframes)
Control	Simple	Complex
Loop	No	Yes (infinite possible)
Direction	Forward/reverse only	Any direction
Best For	Interactions	Automated effects



Transitions For:

- Hover effects
- Click feedback
- Focus states
- Interactive elements

Animations For:

- Page load effects
- Loading indicators
- Background movement
- Attention-grabbers
- Decorative effects

Keyframes Syntax - Defining Animations

Step 1: DEFINE the animation with @keyframes

```
@keyframes animation-name {
    /* Define what happens at each stage */
}
```

Step 2: APPLY it to an element

```
.element {
    animation: animation-name duration;
}
```

```
@keyframes animation-name {
  from { /* starting state */ }
  to { /* ending state */ }
}
/* OR with percentages */
@keyframes animation-name {
  0% { /* start */ }
  50% { /* middle */ }
  100% { /* end */ }
}
```

Simple (from/to):

```
@keyframes name {
    from { /* start */ }
    to { /* end */ }
}
```

Complex (percentages):

```
@keyframes name {
    0% { /* start */ }
    50% { /* middle */ }
    100% { /* end */ }
}
```

Multiple keyframes:

```
@keyframes name {
    0%, 100% { /* same */ }
    50% { /* different */ }
}
```

Syntax Option 1: from/to (Simple)

```css
/* Syntax Option 1: from/to (Simple) */
@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

.hero {
  animation: fadeIn 1s ease-out;
}
```

What it means:

- `from` = Starting state (0%)
- `to` = Ending state (100%)
- Browser animates smoothly between them

Example - Slide In:

```css
/* Example - Slide In: */
@keyframes slideIn {
  from {
    transform: translateX(-100px);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}

.card {
  animation: slideIn 0.5s ease-out;
}
```

Syntax Option 2: Percentages (Complex)

```css
@keyframes bounce {
  0% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-20px);
  }
  100% {
    transform: translateY(0);
  }
}
```

What it means:

- `0%` = Start (same as `from`)
- `50%` = Halfway through
- `100%` = End (same as `to`)
- Can have as many steps as you want!

```css
@keyframes rainbow {
  0% {
    background-color: red;
  }
  20% {
    background-color: orange;
  }
  40% {
    background-color: yellow;
  }
  60% {
    background-color: green;
  }
  80% {
    background-color: blue;
  }
  100% {
    background-color: purple;
  }
}

.element {
  animation: rainbow 5s linear infinite;
}
```

Applying Animations - The Animation Property

```
/* The Shorthand Syntax: */
.element {
  animation: name duration timing-function delay iteration-count direction fill-mode play-state;
}
```

Minimal (Most Common):

```
.element {
animation: name duration;
}
/* Example */
.hero {
animation: fadeIn 1s;
}
```

Breaking Down Each Property:

1. **animation-name (Required)**
2. **animation-duration (Required)**
3. animation-timing-function (Optional)
4. animation-delay (Optional)
5. animation-iteration-count (Optional)
6. animation-direction (Optional)
7. **animation-fill-mode (Optional but Important!)**
8. animation-play-state (Optional)

Breaking Down Each Property:

```
/* 1. animation-name (Required) */
.element {
    animation: fadeIn;
}       /*         ↑
          Name from @keyframes */
/* Must match your @keyframes name exactly! */
```

Must match your @keyframes name exactly!

```
/* 2. animation-duration (Required) */
.element {
    animation: fadeIn 2s;
}           /*         ↑
              Duration in seconds or milliseconds */
```

Common Durations:

- 0.3s - Quick, subtle
- 0.5s - Standard entrance
- 1s - Smooth, noticeable
- 2s - Slow, dramatic

```
/* 3. animation-timing-function */
.element {
    animation: fadeIn 2s ease-in-out;
}                       /*         ↑
                          Easing function */
```

Same as transitions:

- `ease` (default) - Natural
- `linear` - Constant speed
- `ease-in` - Accelerate
- `ease-out` - Decelerate ⭐ Best for entrances
- `ease-in-out` - Smooth both ends

Breaking Down Each Property:

```
/* 4. animation-delay */
.element {
    animation: fadeIn 2s ease-in-out 1s;
}                            /*         ↑
                      Delay in seconds or milliseconds */
```

Useful for staggered animations!

```
/* 5. animation-iteration-count */
.element {
    animation: fadeIn 2s ease-in-out 1s infinite;
}/*                                    ↑
                  Number of times to repeat (or 'infinite') */
```

- **Number: 1, 2, 3, etc.**
- **infinite - Never stops**

```
/* 6. animation-direction */
.element {
    animation: fadeIn 2s ease-in-out 1s infinite alternate;
}/*                                           ↑
                'normal' | 'reverse' | 'alternate' | 'alternate-reverse' */
```

- **normal (default) - Forward**
- **reverse - Backward**
- **alternate - Forward, then reverse, then forward...**
- **alternate-reverse - Reverse, then forward...**

Breaking Down Each Property:

```
/* 7. animation-fill-mode */
.element {
    animation: fadeIn 2s ease-in-out 1s infinite alternate forwards;
}/*                                                    ↑
                                    'none' | 'forwards' | 'backwards' | 'both' */
```

What happens before/after animation:

- none (default) - Returns to original
- forwards - Stays at final state
- backwards - Jumps to first keyframe during delay
- both - Both forwards and backwards

```
/* 8. animation-play-state */
.element {
    animation: fadeIn 2s ease-in-out 1s infinite alternate forwards paused;
}/*                                                          ↑
                                              'running' | 'paused' */
```

- running (default)
- paused - Freeze animation

Fade In Animation - Elegant Entrances

```css
/* Basic Fade In: */
@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

.hero {
  opacity: 0;  /* Start invisible */
  animation: fadeIn 1s ease-out forwards;
}
```

The Effect:

1. Start: opacity: 0 (invisible)
2. Mid: opacity: 0.5 (half visible)
3. End: opacity: 1 (fully visible)

Smooth, elegant appearance!

Fade In Animation - Elegant Entrances

```css
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);  /* Start 20px down */
  }
  to {
    opacity: 1;
    transform: translateY(0);     /* End at normal position */
  }
}

.hero {
  opacity: 0;
  animation: fadeIn 1s ease-out forwards;
}
```

The Effect:
Start: Element 20px down, invisible
↓
Mid: Element 10px down, half visible
↓
End: Element at normal position, fully visible

Fades in WHILE sliding up! Much more dynamic!

Fade In Animation - Variations

```css
/* Fade In from Left: */
@keyframes fadeInLeft {
  from {
    opacity: 0;
    transform: translateX(-30px);  /* Start left */
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}
```

```css
/* Fade In from Right: */
@keyframes fadeInRight {
  from {
    opacity: 0;
    transform: translateX(30px);  /* Start right */
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}
```

```css
/* Fade In from Top: */
@keyframes fadeInDown {
  from {
    opacity: 0;
    transform: translateY(-30px);  /* Start above */
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

```css
/* Fade In + Scale (Zoom): */
@keyframes fadeInScale {
  from {
    opacity: 0;
    transform: scale(0.9);  /* Start smaller */
  }
  to {
    opacity: 1;
    transform: scale(1);    /* End normal size */
  }
}
```

Complete Hero Section Example:

The Effect:
Page loads
→ Hero section fades in
→ 0.2s later: Title fades in
→ 0.4s later: Subtitle fades in
→ 0.6s later: Button fades in

```css
/* Define the animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Apply to hero section */
.hero {
  opacity: 0;  /* Start invisible (important!) */
  animation: fadeIn 1s ease-out forwards;
}

.hero-title {
  opacity: 0;
  animation: fadeIn 1s ease-out 0.2s forwards;  /* Delayed */
}

.hero-subtitle {
  opacity: 0;
  animation: fadeIn 1s ease-out 0.4s forwards;  /* More delayed */
}

.hero-button {
  opacity: 0;
  animation: fadeIn 1s ease-out 0.6s forwards;  /* Most delayed */
}
```

Complete Card Section

Example:

The Effect:

Cards appear one after another!

```css
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.card {
  opacity: 0;
  animation: fadeIn 0.6s ease-out forwards;
}

/* Stagger with nth-child */
.card:nth-child(1) {
  animation-delay: 0s;
}
.card:nth-child(2) {
  animation-delay: 0.1s;
}
.card:nth-child(3) {
  animation-delay: 0.2s;
}
.card:nth-child(4) {
  animation-delay: 0.3s;
}
.card:nth-child(5) {
  animation-delay: 0.4s;
}
.card:nth-child(6) {
  animation-delay: 0.5s;
}
```

Float Animation - Subtle Continuous Movement

```css
@keyframes float {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-10px);
  }
}

.icon {
  animation: float 3s ease-in-out infinite;
}
```

What Happens:

0%: [Icon] ———— Normal position
↓
25%: [Icon] ———— Moving up
↓
50%: [Icon] ———— Top (10px up)
↓
75%: [Icon] ———— Moving down
↓
100%: [Icon] ———— Back to normal
↓
Repeats infinitely! ♾️

When to Use:

- Call-to-action buttons
- Icons and badges
- Scroll-down indicators
- Floating elements
- Background decorations

Smooth up-and-down motion that loops forever!

Float Animation - Breaking Down the Code:

```css
@keyframes float {
  0%, 100% {            /* Start AND end */
    transform: translateY(0);
  }
  50% {                 /* Halfway point */
    transform: translateY(-10px);
  }
}

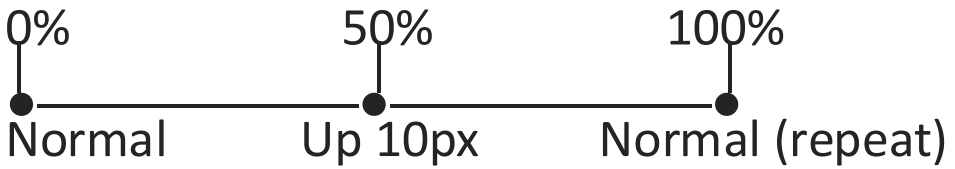
.icon {
  animation: float 3s ease-in-out infinite;
  /*         name  dur timing     loop */
}
```

- 3s - Slow, gentle (try 2s-4s)
- ease-in-out - Smooth acceleration/deceleration
- infinite - Loops forever

Why 0%, 100% together?

- Both start and end at same position
- Creates seamless loop!

The Timeline:



Float Animation - Variations

```css
/* Subtle Float (Recommended): */
@keyframes float {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-10px); }
}

.icon {
  animation: float 3s ease-in-out infinite;
}
/* 10px movement, 3s duration = gentle */
```

```css
/* Dramatic Float: */
@keyframes floatDramatic {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
}

.badge {
  animation: floatDramatic 2s ease-in-out infinite;
}
/* 20px movement, 2s duration = noticeable */
```

```css
/* Slow Drift: */
@keyframes drift {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-5px); }
}

.background-shape {
  animation: drift 5s ease-in-out infinite;
}
/* 5px movement, 5s duration = barely noticeable */
```

Float Animation - Variations

```css
/* Float + Rotate: */
@keyframes floatRotate {
  0%, 100% {
    transform: translateY(0) rotate(0deg);
  }
  50% {
    transform: translateY(-10px) rotate(5deg);
  }
}

.icon {
  animation: floatRotate 3s ease-in-out infinite;
}
/* Floats AND gently rotates! More dynamic! */
```

```css
/* Float + Scale: */
@keyframes floatScale {
  0%, 100% {
    transform: translateY(0) scale(1);
  }
  50% {
    transform: translateY(-10px) scale(1.05);
  }
}

.badge {
  animation: floatScale 3s ease-in-out infinite;
}
/* Grows slightly as it floats up! */
```

Scroll-Down Indicator: Example:

Arrow bounces to indicate scrolling!

```html
<div class="scroll-indicator">
  <span>↓ Scroll Down</span>
</div>
```

```css
@keyframes float {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-10px); }
}

.scroll-indicator {
  position: fixed;
  bottom: 2rem;
  left: 50%;
  transform: translateX(-50%);

  animation: float 2s ease-in-out infinite;
}
```

Pulse Animation - Attention-Grabbing Effect

```css
@keyframes pulse {
  0%, 100% {
    transform: scale(1);
  }
  50% {
    transform: scale(1.05);
  }
}

.notification {
  animation: pulse 2s ease-in-out infinite;
}
```

What Happens:

0%: [Badge] ———— Normal size (100%)
↓
25%: [Badge] ———— Growing
↓
50%: [Badge] ———— Bigger (105%)
↓
75%: [Badge] ———— Shrinking
↓
100%: [Badge] ———— Back to normal
↓
Repeats infinitely! ♾️

When to Use:

- Notification badges (unread count)
- Alert indicators
- "New" labels
- Live status indicators
- Important CTAs (sparingly!)
- Recording indicators

When NOT to Use:

- Multiple elements at once
- Anywhere near body text
- Navigation items
- Regular content
- Anything the user needs to read

Pulse Animation - Breaking Down the Code:

```
@keyframes pulse {
  0%, 100% {              /* Start AND end */
    transform: scale(1);  /* Normal size */
  }
  50% {                   /* Halfway */
    transform: scale(1.05); /* 5% bigger */
  }
}

.notification-badge {
  animation: pulse 2s ease-in-out infinite;
  /*         name dur    timing     loop */
}
```

Why 0%, 100% together?

- Both at scale(1) = seamless loop!

The Scale:

- scale(1) = 100% (normal)
- scale(1.05) = 105% (5% bigger)
- scale(1.1) = 110% (10% bigger)
- scale(0.95) = 95% (5% smaller)

- 2s - Moderate speed (try 1.5s-3s)
- ease-in-out - Smooth breathing effect
- infinite - Never stops

Pulse Animation - Variations

```
/* Subtle Pulse (Recommended): */
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.05); }
}

.badge {
  animation: pulse 2s ease-in-out infinite;
}
/* 5% growth = noticeable but not annoying */
```

```
/* Gentle Pulse: */
@keyframes pulseGentle {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.02); }
}

.live-indicator {
  animation: pulseGentle 3s ease-in-out infinite;
}
/* 2% growth, 3s duration = barely visible, calm */
```

```
/* Dramatic Pulse: */
@keyframes pulseDramatic {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.15); }
}

.urgent-alert {
  animation: pulseDramatic 1s ease-in-out infinite;
}
/* 15% growth, 1s duration = very noticeable */
```

```
/* Reverse Pulse (Shrink): */
@keyframes pulseShrink {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(0.95); }
}

.button {
  animation: pulseShrink 2s ease-in-out infinite;
}
/* Shrinks instead of grows! */
```

Pulse Animation - Variations

```css
/* Pulse + Opacity: */
@keyframes pulseOpacity {
  0%, 100% {
    transform: scale(1);
    opacity: 1;
  }
  50% {
    transform: scale(1.05);
    opacity: 0.8;
  }
}

.alert {
  animation: pulseOpacity 2s ease-in-out infinite;
}
/* Grows and dims slightly = softer effect! */
```

```css
/* Pulse + Color: */
@keyframes pulseColor {
  0%, 100% {
    transform: scale(1);
    background-color: #e74c3c;
  }
  50% {
    transform: scale(1.05);
    background-color: #c0392b;
  }
}

.notification {
  animation: pulseColor 2s ease-in-out infinite;
}
/* Pulses AND changes color! */
```

Notification Badge - Example:

Notification Badge:

Badge pulses to show unread messages!

```html
<div style="position: relative;">
  <button>Messages</button>
  <span class="notification-badge">3</span>
</div>
```

```css
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.1); }
}

.notification-badge {
  position: absolute;
  top: -8px;
  right: -8px;
  background-color: #e74c3c;
  color: white;
  font-size: 0.75rem;
  font-weight: 700;
  padding: 0.25rem 0.5rem;
  border-radius: 12px;

  animation: pulse 2s ease-in-out infinite;
}
```

Product Label - Example:

New Product

```html
<h3>
  Product Name
  <span class="new-label">NEW</span>
</h3>
```

```css
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.05); }
}

.new-label {
  display: inline-block;
  background-color: #2ecc71;
  color: white;
  font-size: 0.75rem;
  font-weight: 700;
  padding: 0.25rem 0.5rem;
  border-radius: 4px;

  animation: pulse 2s ease-in-out infinite;
}
```

Animation Best Practices - Less is More

Animation Best Practices - Don't Overwhelm Users!

The Golden Rules:

1. **Purpose Over Flash** - Every animation needs a reason
2. **Subtlety Wins** - Less movement, more impact
3. **Performance Matters** - Use `transform` and `opacity`
4. **One Star** - Only one element should demand attention
5. **Test on Real Users** - Ask: "Is this annoying?"

When in Doubt: Remove the animation!

Common Mistakes:

- Too Many Animations at Once
- Too Fast
- Too Slow
- Excessive Movement

Animation should:

- Guide user attention
- Provide feedback
- Add delight
- Support the content

Respecting User Preferences – Accessibility First

Respecting User Preferences - Critical Accessibility!

The Problem: Some users experience:

- Motion sickness
- Dizziness
- Nausea
- Vestibular disorders
- Seizures (from rapid animations)

Animations can make sites UNUSABLE for them!

What this does:

- Detects user's OS/browser preference
- Disables animations automatically
- Makes your site accessible!

```css
/* Define animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Apply normally */
.hero {
  opacity: 0;
  animation: fadeIn 1s ease-out forwards;
}

/* Disable for accessibility */
@media (prefers-reduced-motion: reduce) {
  .hero {
    animation: none;
    opacity: 1;  /* Just show it immediately */
    transform: none;
  }
}
```

Respecting User Preferences – Accessibility First

Add this to EVERY project! Not optional!

Why This Matters:

- **Legal requirement** in many places
- **Makes your site accessible** to all users
- **Shows you care** about user wellbeing
- **Simple to implement** - just add the media query
- **Professional standard** - all major sites do this

This is not optional - it's required for accessibility!
Always include @media (prefers-reduced-motion)!

```
/* Your regular animations */
.card {
  animation: slideIn 0.5s ease-out;
}

.button {
  transition: transform 0.3s ease;
}

.icon {
  animation: float 3s ease-in-out infinite;
}

/* ALWAYS include this at the end of your CSS! */
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

PSEUDO-CLASSES & PSEUDO-ELEMENTS



What Are Pseudo-Classes?
- Special States

Pseudo-Classes - Target Elements in Special States

What Are Pseudo-Classes?

- Selectors that target elements in specific **states**
- Use single colon: `:hover`, `:focus`, `:active`
- No extra HTML needed - pure CSS!
- Change styling based on user interaction or position

Why Use Pseudo-Classes?

- **No JavaScript needed** - Pure CSS interactions
- **Better accessibility** - :focus for keyboard users
- **Cleaner HTML** - No extra classes
- **Dynamic styling** - Responds to user state

```css
/* :hover - Mouse is over element */
a:hover {
  color: blue;
  text-decoration: underline;
}

/* User hovers → style changes! */
/* :focus - Element has keyboard focus */
input:focus {
  border-color: #3498db;
  outline: 2px solid #3498db;
}

/* User tabs to element → style changes! */
/* :active - Being clicked/pressed */
button:active {
  transform: scale(0.98);
}

/* User clicks → style changes! */
/* :visited - Link has been clicked before */
a:visited {
  color: purple;
}
```

Common Interactive States:

```css
/* :hover - Mouse is over element */
a:hover {
  color: blue;
  text-decoration: underline;
}

/* User hovers → style changes! */
/* :focus - Element has keyboard focus */
input:focus {
  border-color: #3498db;
  outline: 2px solid #3498db;
}

/* User tabs to element → style changes! */
/* :active - Being clicked/pressed */
button:active {
  transform: scale(0.98);
}

/* User clicks → style changes! */
/* :visited - Link has been clicked before */
a:visited {
  color: purple;
}
```

Structural Pseudo-Classes:

```css
/* Structural Pseudo-Classes: */
/* :first-child - First element */
li:first-child {
  font-weight: bold;
}

/* :last-child - Last element */
li:last-child {
  border-bottom: none;
}

/* :nth-child(n) - Specific position */
li:nth-child(2) {
  color: red;  /* Second item */
}

li:nth-child(odd) {
  background: #f0f0f0;  /* Zebra stripes! */
}
li:nth-child(even) {
  background: white;
}
```

Form States:

```css
/* Form States: */
/* :valid - Valid input */
input:valid {
  border-color: green;
}

/* :invalid - Invalid input */
input:invalid {
  border-color: red;
}

/* :disabled - Disabled element */
button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* :checked - Checked checkbox/radio */
input:checked {
  background-color: blue;
}
```

Navigation Links:

```css
/* Navigation: */
.nav-link {
  color: #2c3e50;
  padding: 0.5rem 1rem;
  text-decoration: none;
  border-radius: 6px;
  transition: all 0.3s ease;
}

/* Hover effect */
.nav-link:hover {
  background-color: rgba(52, 152, 219, 0.1);
  color: #3498db;
}

/* Keyboard focus */
.nav-link:focus {
  outline: 2px solid #3498db;
  outline-offset: 2px;
}

/* Active/current page */
.nav-link:active {
  background-color: rgba(52, 152, 219, 0.2);
}
```

Quick Reference:

```
/* Interactive States */
:hover      /* Mouse over */
:focus      /* Keyboard focus */
:active     /* Being clicked */
:visited    /* Previously clicked link */

/* Structural */
:first-child    /* First element */
:last-child     /* Last element */
:nth-child(n)   /* Specific position */
:nth-child(odd) /* Odd positions */
:nth-child(even) /* Even positions */
```

```
/* Form States */
:valid      /* Valid input */
:invalid    /* Invalid input */
:disabled   /* Disabled */
:checked    /* Checked checkbox/radio */
:required   /* Required field */

/* Other */
:not()      /* Except */
:empty      /* No children */
```

What Are Pseudo-Elements?
- Adding Content with CSS

Pseudo-Elements - Create "Fake" Elements for Styling

What Are Pseudo-Classes?

- Create virtual elements that don't exist in HTML
- Use double colon: ::before, ::after
- Perfect for decorative content
- Keeps HTML clean and semantic

Why Use Pseudo-Classes?

- Clean HTML - No extra markup needed
- Easy to maintain - Change CSS, not HTML
- Decorative only - Keeps content semantic
- Powerful styling - Full CSS control

```
/* The Pattern: */
selector::pseudo-element {
  content: "...";  /* Required! */
  /* styles */
}
```

```
/* Pseudo-Classes vs Pseudo-Elements: */
/* Pseudo-CLASS (single colon) - STATE */
a:hover { }
/* Targets existing element in a state */

/* Pseudo-ELEMENT (double colon) - CONTENT */
a::before { }
/* Creates new virtual element */
```

Pseudo-Elements

Quick Reference:

```
::before      /* Before element content */
::after       /* After element content */
::first-letter   /* First letter */
::first-line     /* First line */
::selection      /* Text selection */
::placeholder    /* Input placeholder */
```

::before - Inserts content BEFORE element

::after - Inserts content AFTER element

```
/* The Big Two: ::before and ::after */
/* ::before - Inserts content BEFORE element */
.element::before {
  content: "→ ";
}

/* ::after - Inserts content AFTER element */
.element::after {
  content: " ←";
}
```

```
<p class="element">Hello</p>
Renders as: → Hello ←
```

```css
/* Adding Decorative Quotes: */
.quote {
  position: relative;
  padding: 2rem;
  font-size: 1.25rem;
  font-style: italic;
}

.quote::before {
  content: '"';
  font-size: 4rem;
  color: #3498db;
  position: absolute;
  top: -10px;
  left: 10px;
  line-height: 1;
}

.quote::after {
  content: '"';
  font-size: 4rem;
  color: #3498db;
  position: absolute;
  bottom: -30px;
  right: 10px;
  line-height: 1;
}
```

::before and ::after - Decorative Content

::before and ::after - Your Go-To Pseudo-Elements

Result: Large decorative quotes around text!

```html
<blockquote class="quote">
  This is an inspiring quote that needs decorative quotation marks.
</blockquote>
```

::before and ::after - Decorative Content

::before and ::after - Your Go-To Pseudo-Elements

```css
/* Icon Before Text: */
.link::before {
  content: "→ ";
  color: #3498db;
  font-weight: bold;
  margin-right: 0.5rem;
}
```

Result: → Read more

```
<a href="#" class="link">Read more</a>
Result: → Read more
```

Decorative Underlines - Animated Link Effect

```css
/* The Effect: Underline slides in from left on hover! */
.link {
  position: relative;
  color: #3498db;
  text-decoration: none;
  padding-bottom: 2px;
}

.link::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  width: 0;    /* Start invisible */
  height: 2px;
  background-color: #3498db;
  transition: width 0.3s ease;
}

.link:hover::after {
  width: 100%; /* Expand to full width! */
}
```

Animated Underline Effect - Classic Professional Pattern

How It Works:

- Remove default underline with text-decoration: none
- Position link relative for absolute child
- Create ::after with empty content: ''
- Position it absolutely at bottom
- Start with width: 0 (invisible)
- Add transition for smooth animation
- On hover, expand to width: 100%

```html
<a href="#" class="link">Hover over me</a>
```

Section Dividers - Decorative Lines with ::after

```css
/* Centered Decorative Line: */
.section-title {
  text-align: center;
  font-size: 2.5rem;
  margin-bottom: 3rem;
}

.section-title::after {
  content: '';
  display: block;
  width: 50px;
  height: 3px;
  background-color: #3498db;
  margin: 1rem auto 0;
}
```

Section Dividers - Clean Visual Separation

```html
<h2 class="section-title">Our Services</h2>
```

TRANSFORM



The Transform Property – Manipulate Elements

Transform changes how element LOOKS, not where it IS in layout!

What is Transform?

- **Move**, **scale**, **rotate**, and **skew** elements
- Changes visual appearance without affecting layout
- Other elements don't move when you transform!
- Perfect for hover effects and animations

Key Benefits:

- **Hardware accelerated** - GPU handles it
- **Smooth animations** - 60fps performance
- **No layout reflow** - Other elements stay put
- **Combine multiple** - Stack effects together

```
/* Basic Syntax: */
.element {
  transform: function(value);
}
```

Transform Functions - The Building Blocks

1. Translate - Move Elements

```css
/* Move horizontally (right is positive) */
transform: translateX(50px);

/* Move vertically (down is positive) */
transform: translateY(-20px);

/* Move both at once */
transform: translate(50px, -20px);
```

2. Scale - Resize Elements

```css
/* Scale both directions */
transform: scale(1.2);     /* 120% size */
transform: scale(0.5);     /* 50% size */

/* Scale separately */
transform: scaleX(1.5);    /* Wider */
transform: scaleY(0.8);    /* Shorter */
transform: scale(1.5, 0.8); /* Both */
```

Transform Functions - The Building Blocks

3. Rotate - Spin Elements

```css
/* Clockwise rotation */
transform: rotate(45deg);

/* Counter-clockwise */
transform: rotate(-45deg);

/* Full rotation */
transform: rotate(360deg);
```

4. Skew - Slant Elements

```css
/* Skew horizontally */
transform: skewX(10deg);

/* Skew vertically */
transform: skewY(10deg);

/* Skew both */
transform: skew(10deg, 5deg);
```

Units:

- deg = degrees (most common)
- turn = full rotations (1turn = 360deg)
- rad = radians

Quick Reference:

- Translate = move,
- Scale = size,
- Rotate = turn!

```css
/* TRANSLATE (Move) */
transform: translateX(50px);      /* Right */
transform: translateX(-50px);     /* Left */
transform: translateY(20px);      /* Down */
transform: translateY(-20px);     /* Up */
transform: translate(50px, 20px); /* Both */

/* SCALE (Size) */
transform: scale(1.2);     /* Bigger */
transform: scale(0.8);     /* Smaller */
transform: scaleX(1.5);    /* Wider */
transform: scaleY(0.5);    /* Shorter */

/* ROTATE (Spin) */
transform: rotate(45deg);  /* Clockwise */
transform: rotate(-45deg); /* Counter-clockwise */

/* SKEW (Slant) */
transform: skewX(10deg);   /* Horizontal slant */
transform: skewY(10deg);   /* Vertical slant */
```

Combining Transforms - Multiple Effects at Once

Multiple Transforms Together:

```css
.card:hover {
  transform:
    translateY(-10px)
    rotate(2deg)
    scale(1.05);
}
```

This does THREE things at once:

1. Lifts up 10px
2. Rotates 2 degrees
3. Grows 5% bigger

The Syntax:

```css
/* Space-separated, NO commas! */
transform: function1() function2() function3();

/* Correct */
transform: translateY(-10px) scale(1.1);

/* Wrong - no commas! */
transform: translateY(-10px), scale(1.1);
```

Order Matters!

```css
/* Rotate THEN translate */
transform: rotate(45deg) translateX(100px);

/* Translate THEN rotate */
transform: translateX(100px) rotate(45deg);
```

Common Combinations:

```css
/* Card Hover (Lift + Shadow feel): */
.card:hover {
  transform: translateY(-8px) scale(1.02);
}

/* Playful Rotation: */
.image:hover {
  transform: scale(1.1) rotate(3deg);
}

/* Button Press: */
.button:active {
  transform: translateY(2px) scale(0.98);
}

/* Icon Attention: */
.icon:hover {
  transform: scale(1.2) rotate(10deg);
}
```

```css
/* Real Example: */
.card {
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.card:hover {
  transform: translateY(-10px) rotate(1deg) scale(1.02);
  box-shadow: 0 12px 24px rgba(0, 0, 0, 0.15);
}
```

Transform for Hover Effects - Bread and Butter Patterns

1. The Lift Effect

```css
.card {
  transition: transform 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
}
```

2. Scale Up (Zoom)

```css
.image {
  transition: transform 0.3s ease;
}

.image:hover {
  transform: scale(1.1);
}
```

1. The Rotate Effect

```css
.icon {
  transition: transform 0.3s ease;
}

.icon:hover {
  transform: rotate(5deg);
}
```

4. Lift + Scale (Premium Combo)

```css
.card {
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.card:hover {
  transform: translateY(-8px) scale(1.02);
  box-shadow: 0 12px 24px rgba(0, 0, 0, 0.15);
}
```

Transform for Hover Effects - Bread and Butter Patterns

5. Icon Spin

```css
.refresh-icon {
  transition: transform 0.5s ease;
}

.refresh-icon:hover {
  transform: rotate(180deg);
}
```

6. Link Arrow Slide

```css
.link {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
}

.link .arrow {
  transition: transform 0.3s ease;
}

.link:hover .arrow {
  transform: translateX(5px);
}
```

Quick Reference:

```css
/* Lift */
.element:hover { transform: translateY(-5px); }

/* Zoom */
.element:hover { transform: scale(1.1); }

/* Tilt */
.element:hover { transform: rotate(5deg); }

/* Combo */
.element:hover { transform: translateY(-5px) scale(1.05); }

/* Always add transition! */
.element { transition: transform 0.3s ease; }
```

Button Press Effect

1. The Lift Effect

```css
.button {
  transition: transform 0.1s ease;
}

.button:active {
  transform: scale(0.95);
}
```

What Happens:

- Normal: [Button] (100%)
- Click: [Button] (95% - shrinks!)
- Release: [Button] (100% - back)

Why It Works:

- `:active` triggers when mouse button is down
- `scale(0.95)` shrinks slightly (5% smaller)
- Fast transition (0.1s) = instant feedback
- Mimics physical button depression

Complete Button Example:

```css
.button {
  background-color: #3498db;
  color: white;
  padding: 1rem 2rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;

  /* Smooth transitions */
  transition: transform 0.1s ease,
              background-color 0.3s ease;
}

/* Hover - lift up */
.button:hover {
  background-color: #2980b9;
  transform: translateY(-2px);
}

/* Click - press down */
.button:active {
  transform: scale(0.95);
}
```


CSS custom properties?
Transform effects ?
Smooth transitions?
The first PROJECT?
Subtle animations?
Anything unclear?
HAVE ANY
QUESTIONS?